Learn how to build APIs in .NET with our **Ultimate ASP.NET Core Web API!**

Angular Material Table, Filtering, Sorting, Paging

Posted by **Marinko Spasojevic** | Updated Date Jan 20, 2021 | **45**



Want to build **great APIs?** Or become **even better** at it? Check our program **Ultimate ASP.NET Core Web API** and learn how to create a full production-ready ASP.NET Core API using only the **latest .NET technologies**. Bonus materials included!

We are going to divide this article into two major parts. The **first part** will consist of creating environment files, HTTP repository service, and creating a new Owner module with the lazy loading feature. As you can see, everything is Angular specific, so we won't dive too deep into these sections. We already have **Angular Series** in which we have talked about these topics in great detail. So if you are not familiar with these topics, we strongly recommend reading the mentioned series.

In our source code, we can find the `OwnerAccountServer` folder which contains the entire .NET Core project, which we have created in **.NET Core Series**. In



--- **FREE** eBook ---
Top **16 BEST PRACTIC**
to improve API effective
10x.
Find out how!

EMAIL ADDRESS

YOUR NAME

SUBSCRIBE

the same folder, we can find the `_MySQL_Init_Script` folder which contains a script to create a MySQL database with its tables. Just run that script in the MySQL database and you are ready to go.

The second part will consist of creating a material table and populating that table with data from our server. Furthermore, we are going to create the filter, sorting, and paging functionalities for that table.

So, it's time to start our job.

▸ THIS ARTICLE IS PART OF THE SERIES

For the complete navigation and all the basic instructions of the Angular Material series, check out: **Introduction of the Angular Material series.**

The source code is available at GitHub **Angular Material Table – Source Code**

We are going to divide this post into several sections:

- Environment, HTTP and Owner Module
  - Creating a New Owner Module
- Using Material Table to Display Data
- Sorting Data in Material Table
- Filter Functionality in Material Table
- Paging Functionality
- Conclusion

Environment, HTTP and Owner Module

Let's start with the environment file modifications.

We are going to modify the `environment.prod.ts` file first:

```
export const environment = {
  production: true,
  urlAddress: 'http://www.ang-material-account-own
};
```

After that, let's modify the `environment.ts` file:

```
export const environment = {
  production: false,
```

```
  urlAddress: 'http://localhost:5000'
};
```

Having these environment files modified, it is time to create a service for sending the HTTP requests towards our server.

To do that, we are going to create a service file first:

```
ng g service shared/repository --skipTests
```

After creation, we have to modify that file:

```
import { Injectable } from '@angular/core';
import { HttpClient, HttpHeaders } from '@angular/
import { environment } from './../../environments/

@Injectable({
  providedIn: 'root'
})
export class RepositoryService {

  constructor(private http: HttpClient) { }

  public getData = (route: string) => {
    return this.http.get(this.createCompleteRoute(
  }

  public create = (route: string, body) => {
    return this.http.post(this.createCompleteRoute
  }

  public update = (route: string, body) => {
    return this.http.put(this.createCompleteRoute(
  }

  public delete = (route: string) => {
    return this.http.delete(this.createCompleteRou
  }

  private createCompleteRoute = (route: string, en
    return `${envAddress}/${route}`;
  }

  private generateHeaders = () => {
    return {
      headers: new HttpHeaders({'Content-Type': 'a
    }
  }
}
```

Excellent. We have prepared our service file. If you want to learn more about environment files, services, and HTTP, you can read that in the **Angular Series Article** which covers all of these topics.

One more thing that we need to do is to register `HttpClientModule` in the `app.module.ts` file:

```
import { HttpClientModule } from '@angular/common/


imports: [
    …
    HttpClientModule
  ],
```

Creating a New Owner Module

Let's create a new Owner module, and the routes for that module as well:

```
ng g module owner --module app
```

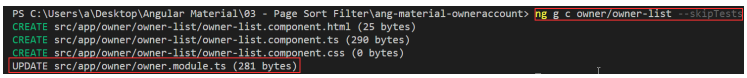
We are going to register this module into the main routing module but in such a way to support the lazy loading feature:

```
const routes: Routes = [
  { path: 'home', component: HomeComponent},
  { path: 'owner', loadChildren: () => import('./o
  { path: '', redirectTo: '/home', pathMatch: 'ful
];
```

To read more about multiple modules and lazy loading in Angular, you can visit an article on the following link **Lazy Loading in Angular**.

Right now, we have to create a new component to show the list of all the owners from the database:

```
ng g component owner/owner-list --skipTests
```



We need to have routing for the components inside this module, so let's create a new routing module for the Owner module components:

```
ng g module owner/owner-routing --module owner
```

And let's modify that module file:

```
import { NgModule } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Routes, RouterModule } from '@angular/rou
import { OwnerListComponent } from '../owner-list/

const routes: Routes = [
  { path: 'owners', component: OwnerListComponent
```

```
];

@NgModule({
  imports: [
    CommonModule,
    RouterModule.forChild(routes)
  ],
  exports: [
    RouterModule
  ],
  declarations: []
})
export class OwnerRoutingModule { }
```

Finally, to make all this to work, we need to modify our routes in the `sidenav-list.component.html` file:

```
<a mat-list-item routerLink="/owner/owners" (click
   <mat-icon>assignment_ind</mat-icon> <span class
</a>
```

And the `header.component.html` file:

```
<li>
    <a routerLink="/owner/owners">Owner Actions</a
</li>
```

That is it. We can confirm now that our routing settings work as it supposed to:



Excellent. Right now, we can dedicate our work to fetch some data from the database and show them in the material table component.

Using Material Table to Display Data

Because we have created another module in our Angular app, we need to import the `Material module` file inside the `owner.module.ts` file:

```
import { MaterialModule } from './../material/mate


imports: [
    …
    MaterialModule
  ],
```

Once we create the Shared module, we will fix this code repetition (MaterialModule inside the App module and Owner module).

For now, let's continue by creating the `_interface` folder and inside it the `owner.model.ts` file:

```
export interface Owner{
    id: string;
    name: string;
    dateOfBirth: Date;
    address: string;
}
```

Because we want to use the material table component, we need to register its own module in the `material.module.ts` file:

```
import { MatTableModule } from '@angular/material/


imports: [
    MatTableModule,

exports: [
    MatTableModule,
```

Then, let's modify the `owner-list.component` file:

```
<table mat-table [dataSource]="dataSource">
  <ng-container matColumnDef="name">
    <th mat-header-cell *matHeaderCellDef> Name </
    <td mat-cell *matCellDef="let element"> {{elem
  </ng-container>

  <ng-container matColumnDef="dateOfBirth">
    <th mat-header-cell *matHeaderCellDef> Date of
    <td mat-cell *matCellDef="let element"> {{elem
  </ng-container>

  <ng-container matColumnDef="address">
    <th mat-header-cell *matHeaderCellDef> Address
    <td mat-cell *matCellDef="let element"> {{elem
  </ng-container>

  <ng-container matColumnDef="details">
    <th mat-header-cell *matHeaderCellDef> Details
    <td mat-cell *matCellDef="let element">
      <button mat-icon-button color="primary" (cli
```

```
            <mat-icon class="mat-18">reorder</mat-ic
        </button>
      </td>
    </ng-container>

    <ng-container matColumnDef="update">
        <th mat-header-cell *matHeaderCellDef> Updat
        <td mat-cell *matCellDef="let element">
          <button mat-icon-button color="accent" (cl
              <mat-icon class="mat-18">system_update
          </button>
        </td>
      </ng-container>

      <ng-container matColumnDef="delete">
          <th mat-header-cell *matHeaderCellDef> Del
          <td mat-cell *matCellDef="let element">
            <button mat-icon-button color="warn" (cl
                <mat-icon class="mat-18">delete</mat
            </button>
          </td>
        </ng-container>

    <tr mat-header-row *matHeaderRowDef="displayedCo
    <tr mat-row *matRowDef="let row; columns: displa
  </table>
```

The `mat-table` element transforms this table into a material one. With the `dataSource` attribute, we provide a data source for our table. Inside every `ng-container` tag, we define the column definition and the value to be displayed. It is very important to match the `matColumnDef` value with the property name of our `Owner` interface.

Finally, in the last two `tr` tags, we define an order for our header columns and the row definitions. So, what we need to do right now is to create our `datasource` and `displayedColumns` properties in the `ownerlist.component.ts` file:

```
import { RepositoryService } from './../../shared/
import { Component, OnInit } from '@angular/core';
import { MatTableDataSource } from '@angular/mater
import { Owner } from '../../_interface/owner.mode

@Component({
  selector: 'app-owner-list',
  templateUrl: './owner-list.component.html',
  styleUrls: ['./owner-list.component.css']
})
export class OwnerListComponent implements OnInit

  public displayedColumns = ['name', 'dateOfBirth'
];
  public dataSource = new MatTableDataSource<Owner
```

```
  constructor(private repoService: RepositoryServi

  ngOnInit() {
    this.getAllOwners();
  }

  public getAllOwners = () => {
    this.repoService.getData('api/owner')
    .subscribe(res => {
      this.dataSource.data = res as Owner[];
    })
  }

  public redirectToDetails = (id: string) => {

  }

  public redirectToUpdate = (id: string) => {

  }

  public redirectToDelete = (id: string) => {

  }

}
```

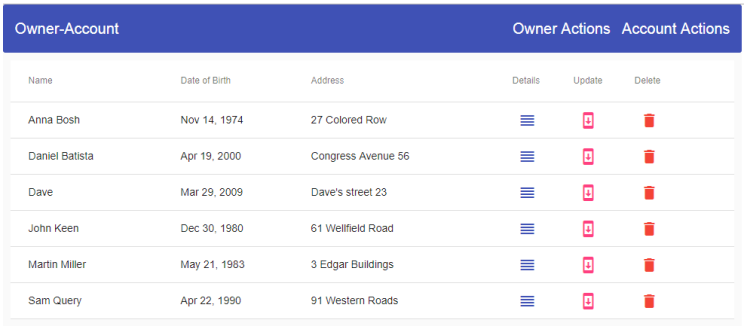
If we change the order of elements inside the `displayedColumns` array, it will change the order of the columns inside our table.

Right now, if we start our application and navigate to the Owner Actions menu, we are going to see a populated material table. But we are missing some styles, so let's add those in the `owner-list.component.css` file:

```
table {
    width: 100%;
    overflow-x: auto;
    overflow-y: hidden;
    min-width: 500px;
}

th.mat-header-cell {
    text-align: left;
    max-width: 300px;
}
```

Now we should have a better-styled table:



Sorting Data in Material Table

We want to add the sorting functionality to our table, and for that purpose, we are going to use the `matSort` directive on the `table` tag. Moreover, we need to place the `mat-sort-header` directive for each header cell that will trigger sorting.

So, let's do that now.

Modifying the `table` tag is going to be our first task:

```
<table mat-table [dataSource]="dataSource" matSort
```

Then, we are going to add the `mat-sort-header` directive to the `Name`, `DateOfBirth`, and `Address` tags:

```
<th mat-header-cell *matHeaderCellDef mat-sort-hea
...

<th mat-header-cell *matHeaderCellDef mat-sort-hea
...

<th mat-header-cell *matHeaderCellDef mat-sort-hea
```

To make sorting functionality up and running, we need to modify the `owner-list.component.ts` file as well:

```
import { Component, OnInit, ViewChild, AfterViewIn
import { MatSort } from '@angular/material/sort';
...
export class OwnerListComponent implements OnInit,

  public displayedColumns = ['name', 'dateOfBirth'
  public dataSource = new MatTableDataSource<Owner

  @ViewChild(MatSort) sort: MatSort;

  constructor(private repoService: RepositoryServi

  ngOnInit() {
    this.getAllOwners();
  }

  ngAfterViewInit(): void {
```

```
    this.dataSource.sort = this.sort;
  }
  .
  .
  .
```

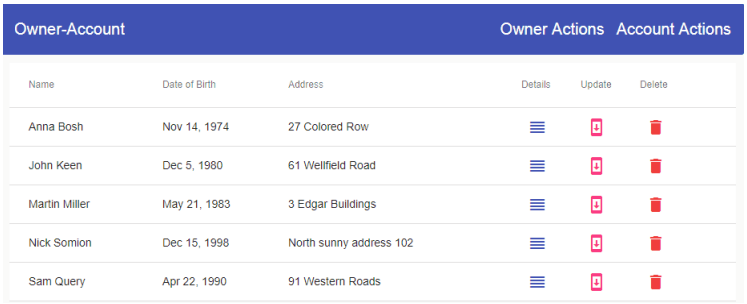
Lastly, we need to add the `MatSortModule` inside of the `material.module.ts` file:

```
import { MatSortModule } from '@angular/material/s


imports: [
    MatSortModule,

exports: [
    MatSortModule,
```

Now, we can check our result:



By default, sorting starts with ascending order first and then descending. We can change that behavior by adding the `matSortStart` attribute to `desc` next to the `matSort` directive:

```
<table mat-table [dataSource]="dataSource" matSort
```

If we don't want to use `MatTableDataSource` for sorting, but to provide our own sorting logic, we can use the `(matSortChange)` event to receive the active sorting column and the sorting order as well:

```
<table mat-table [dataSource]="dataSource" matSort
```

Once we click on the name column it will generate the following JSON object (Of course, don't forget to add the function in the .ts file):

```
{active: "name", direction: "asc"}
```

```
1. active:"name"
2. direction:"asc"
3. __proto__:Object
```

Filter Functionality in Material Table

For this functionality, we need to provide our own input field and a custom function to filter our data. Only then, we can use `MatTableDataSource`'s filter property. To implement filtering, we are going to add the following code right above our table in the HTML file:

```
<div fxLayout fxLayoutAlign="center center">
  <mat-form-field fxFlex="40%">
    <input matInput type="text" (keyup)="doFilter(
  </mat-form-field>
</div>
```

And then to write the following function in the component file:

```
public doFilter = (value: string) => {
    this.dataSource.filter = value.trim().toLocale
  }
```

Finally, because we are using the `matInput` directive to transform regular input into the material input field, we need to register its modules inside the `material.module.ts` file:

```
import { MatFormFieldModule } from '@angular/mater
import { MatInputModule } from '@angular/material/


imports: [
    MatFormFieldModule,
    MatInputModule,

exports: [
    MatFormFieldModule,
    MatInputModule,
```

As we can see from the HTML file, we are using the `fxLayout` directive. But, because this component is part of a new Owner module, we need to import `FlexLayoutModule` into the Owner module file as well:

```
import { FlexLayoutModule } from '@angular/flex-la

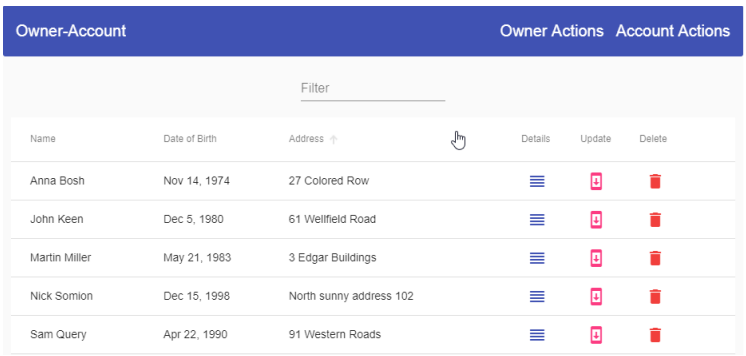

imports: [
    ...
    FlexLayoutModule
```

```
],
```

Of course, this code repetition will be solved as well as soon as we create a Shared module.

Excellent.

Now we can inspect the result:



Paging Functionality

To implement paging with a material table, we need to use a `<mat-paginator>` bellow our table. So, let's start implementation by adding `MatPaginatorModule` inside the `Material` module:

```
import {MatPaginatorModule } from '@angular/materi


imports: [
    MatPaginatorModule,

exports: [
    MatPaginatorModule,
```

Then, let's add `mat-paginator` inside the HTML file:

```
<mat-paginator [pageSize]="2" [pageSizeOptions]="[
</mat-paginator>
```

And finally, let's modify the `owner-list.component.ts` file:

```
import { MatPaginator } from '@angular/material/pa

...

  @ViewChild(MatPaginator) paginator: MatPaginator

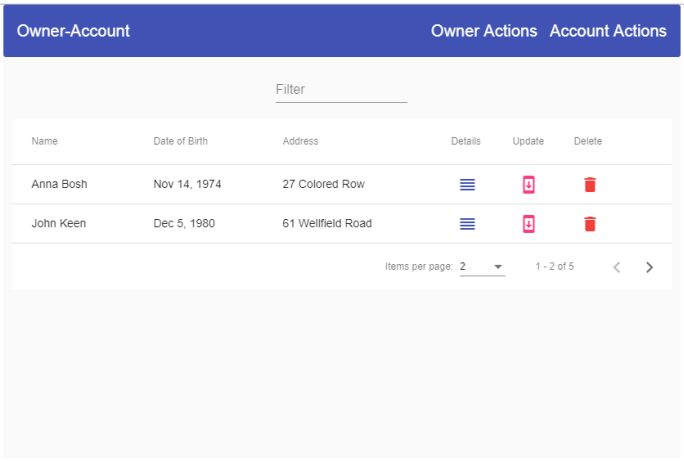
  constructor(private repoService: RepositoryServi

  ngOnInit() {
```

```
    this.getAllOwners();
  }

  ngAfterViewInit(): void {
    this.dataSource.sort = this.sort;
    this.dataSource.paginator = this.paginator;
  }
  ...
```

After these changes, we should have the following result:



If we want to write our custom pagination logic, we should use the `(page)` output event:

```
<mat-paginator [pageSize]="2" [pageSizeOptions]="[
</mat-paginator>
```

For the custom pagination logic, you will have to write a pagination logic on the Web API part. We have a great article that explains **How to Write Paging in ASP.NET Core Web API**. So, feel free to read it and acquire a better knowledge of the server-side paging as well.

Conclusion

So, that's it. Now we have our material table with all the features like sorting, paging, and filtering data.

In this article, we have learned:

- How to use Environment files, HTTP client module, and Lazy Loading feature
- To create a material table
- To apply sorting, filtering, and pagination to the material table

In the next article, we are going to **create error pages by focusing on the material components** and to create owner

details component.



Want to build **great APIs?** Or become **even better** at it? Check our program **Ultimate ASP.NET Core Web API** and learn how to create a full production-ready ASP.NET Core API using only the **latest .NET technologies**. Bonus materials included!

SHARE:

RELATED POSTS



Angular Material – Error and Details Pages With Material Components

Updated Date Jan 27, 2021



Angular Material Form Validation, Input, Datepicker and Modal

Updated Date Aug 24, 2020



Angular Material Navigation Menu – Complete Responsive Navigation

Updated Date Aug 24, 2020



Getting Started With Angular Material

Updated Date Jan 20, 2021

What do you think?

35 Responses

Upvote Funny Love Surprised

Angry Sad

48 Comments Code Maze Privacy Policy Login

Recommend 3 Tweet Share Sort by Best

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS

Name

prabhat genji • 2 years ago

suppose i want to sort data based on from date to date how to implement in filter.

3 | 1 • Reply • Share ›



عبد الكريم بشيري • a year ago

It's very clear, thank you.
juste Viewchild in angular 8 except two parameters: @ViewChild(MatSort, {static: true}) sort: MatSort;

1 | • Reply • Share ›



Marinko Mod ➚ عبد الكريم بشيري • a year ago

Thank you a lot. You are right for Angular 8. I've already replied in comment section, regarding this topic. Anyhow, thank you for reading and commenting as well. Best regards.

| • Reply • Share ›



Vimod • a year ago

How to bind length to pagination?

1 | • Reply • Share ›



Marinko Mod ➚ Vimod • a year ago

I'm not sure, that I understand you. But the length of the source data is already attached to the dataSource object, that is used for Material table.

| • Reply • Share ›



Vimod ➚ Marinko • a year ago

I'm not returning every record from the server-side, for ex if there are 1000 records, I'm pulling only 10 at a time but need to display 1000 total records, so data source object won't work.

| • Reply • Share ›



Marinko Mod ➚ Vimod • a year ago

Well then you have to customize paginator maybe. For sure you have to return from the backend how many rows you have totally, so you could have that info on the client side. We have a link to the server side pagination in this article (at the end of it) so you could check it out to see how we did it.

| • Reply • Share ›



Vimod ➚ Marinko • a year ago

I'm sending total size from server-side, but it is getting override in client-side by 10 (datasource length)

^ | v • Reply • Share ›



Marinko Mod ➚ Vimod • a year ago

Ok. I get it now. So, by default, the datasource.data.length property will be set by the number of items you are paginating at the moment. Which is 10 in your case. But you said that you return the total number of items as well. To apply that value, you have to modify the html first by adding the length attribute like this:

```
<mat-paginator [pagesize]="2" [pagesizeoptions]="[2,
4, 6, 10, 20]" [length]="length">
</mat-paginator>
```

Then you have to create that property in the .ts file and apply it a proper value:

```
public length;
.
.
.
this.length = your number of items
```

This will work for sure.
I hope this helps, just confirm that to me. Best regards.

1 ^ | v • Reply • Share ›



Vimod ➚ Marinko • a year ago

"this.length = your number of items" doesnt work, if I remove "this.dataSource.paginator = this.paginator;" then "this.length = your number of items" will work, but don't know whether it is correct or not.

2 ^ | v • Reply • Share ›



Marinko Mod ➚ Vimod • a year ago

Well let me put it this way. I've used the example from this article (the same source code) and added the code I sent you in the previous comment, and it worked like a charm. You have to have dataSource.paginator initialized, don't remove it. Again pay attention to my comment, did you change the html as well? You have to modify the <mat-paginator> component first by seting the length decorator. Then you have to modify the .ts file by creating the length property and populating it. As I said, I tried this on the source code and it worked as it supposed to.

^ | v • Reply • Share ›



mc alex • 2 months ago

Hi Marinko,
Awesome job explaining this.
A couple of things, 1) when you create the ownler/owner-list component, you use slightly different syntax on the instruction and the screenshot ('--spec=false' vs '--skipTests'). I noticed in an earlier tutorial (https://code-maze.com/get-s... that you have them the other way 'round when you create the Home component file structure, 2) In the filtering functionality area, you have the 'MatInputModule' imported twice - the second one is correct. 3) When you add sorting, you don't mention that the OwnerListComponent now needs to import 'AfterViewInit' from '@angular/core'
Also, a question: Can i implement sorting-descending on only one column (ie the date column) with 'MatSortStart' without writing a custom sorter or is sort order set for the whole table?
Many thanks for your tutorials.

Reply • Share ›

Marinko Mod ↗ mc alex • 2 months ago

Hello Alex. First of all, thanks for the kind words. And thank you very much for reading our articles and for your suggestions.

Regarding those suggestions, you are all corect, I'll fix that ASAP. I was updating entire series and those slipped somehow.

Finally, about your question.

This is from official documentation: To reverse the sort order for all headers, set the matSortStart to desc on the matSort directive. To reverse the order only for a specific header, set the start input only on the header instead.

So, the first part you already know from this article, but for the second part (the one you asked for) you should be using the start attribute in the header it self:

<table matsort="" (matsortchange)="sortData($event)">
<tr>

see more

Reply • Share ›

Rafał Jaworowski • a year ago

Great tutorial!

Reply • Share ›

Marinko Mod ↗ Rafał Jaworowski • a year ago

Thank you very much.

Reply • Share ›

Saddam Hussain • a year ago • edited

Very nice.. Good job.
Saved my time.
Thanks

Reply • Share ›

Marinko Mod ↗ Saddam Hussain • a year ago

Thank you. I am glad you liked it.

Reply • Share ›

Muhamed Hassan • a year ago • edited

thanks for clear tutorial , but can I ask you question about how can I add empty row to angular material table and make table inline editable table ? @Marinko

Reply • Share ›

BalaKrishnan G • a year ago

Super Marinko Spasojevic . Good explanation. Thank you

Reply • Share ›

Rathnakar • a year ago

can you please send me this source code

Reply • Share ›

Marinko Mod ➚ Rathnakar • a year ago

The link is at the beginning of this article. Just read it from the start.

Reply • Share ›

James Cubeta • 2 years ago

An amazingly succinct, complete, and helpful article. Thank you!

Reply • Share ›

Marinko Mod ➚ James Cubeta • 2 years ago

You are very welcome James. I hope you will enjoy other articles as well. Best regards.

Reply • Share ›

Jason Liveston • 2 years ago

Great article, really appreciate the detail and time you put into it. Many articles leave out a lot of what everyone does and only provides demos that are not real world. A few suggestions for future enhancements to this article. You could add logic where the sorting and paging are actually handled from the server code. An example of a real world scenario would be, you have a database with 20K records. You want to display a grid that allows you to display the grid with those 20K records and allow pagination and sorting and searching. You would need to modify your approach as currently you are expecting all 20K records to come down and then you are sorting and paging on the client which would not be a good user experience. Instead if you provided in the service call the sort column, the page size and how many skip and take then the heavy lifting would be performed with the server side code and the results would be filtered back down. Again just ideas for future enhancements.

Reply • Share ›

Marinko Mod ➚ Jason Liveston • 2 years ago

Hello Jason. Thank you for reading this article and for your suggestion. It makes perferc sense, just not for this series because it would have to dive a lot in the .NET Core side, which is not the topic of the series. But we are preparing some new stuff and WEB API searching, sorting and filtering will be included as well. As you can see, I have tried to help in this article a bit, by showing what angular provides for us through this components, if we want manually to sort our data or do the pagination.

Reply • Share ›

Javier Moreno • 2 years ago

this is great! im using angular 8.2 and this help a lot, but sorting is not working , does not throw any error, but its not sorting at all, dont know why

Reply • Share ›



Marinko Mod → Javier Moreno • 2 years ago

Hello Javier. I believe you have a problem due to Angular 8 feature with @VIewChild directives. Try like this:
@ViewChild('MatSort', {static: false}) sort: MatSort;
@ViewChild('MatPaginator', {static: false}) paginator: MatPaginator;
Angular 8 is now expecting 2 arguments

Angular developers announced that this shold be removed in version 9 but for now it has to be applied.

Reply • Share ›

Vasily Asur • 2 years ago

Hello, Marinko. This is another great lesson from the master, thank you. Can you advise what is the best way to display many columns (>5) in a table? The app will mostly be used on a mobile device, but also on a big screen too. Which option is preferable, for example, many columns and a fixed column or several rows in each cell? In the mobile app, I did a few rows in each cell, it was convenient and looked good, but is it possible to use it in mat-table to keep the filter, sorting and pagination?

Reply • Share ›



Marinko Mod → Vasily Asur • 2 years ago

Well that depends on many factors, one of them being the customer you are developing your app for. But as you said, it is up to you to try different versions and to see what fits the best. Usually mobile devices have problems with large tables and sometimes the problem is solved by using a scroll under the table (as bootstrap does with table-responsive class). In some cases I have used the fixed size columns with wrapped text inside it, but again this depends on the content you are showing in those cells as well. So, all I can say is play with it and keep what fits you the best.

1 Reply • Share ›

Vasily Asur → Marinko • 2 years ago

ok, thx for reply, I'll play with it

Reply • Share ›

Kyle D • 2 years ago

FYI : If you are using Angular V8:
@owner-list.component.ts, change @Viewchild(matSort) sort : matsort to this:
@ViewChild('MatSort', {static: false}) sort: MatSort;
@ViewChild('MatPaginator', {static: false}) paginator: MatPaginator;
Angular 8 is now expecting 2 arguments :)).

Reply • Share ›



Marinko Mod → Kyle D • 2 years ago

Thank you Kyle. Yes with Angular v8 ViewChild directives expects

that additional parameter. In Angular v9 this is going to be a default behavior. Best regards.

Reply • Share ›

Kyle D ➚ Marinko • 2 years ago

Haha. I'm doing step by step. Just want to let you know, and if someone have same problem. In the other words, you got a free tester.
Btw, I have an issue with pagination, wish you could help me out.
I construct my paginatoin like this in the .ts file:

```
pageSize = 10;
pageSizeOptions: number[] = [10, 25, 50, 100];
pageEvent: PageEvent;
```

then in the html, I do this :

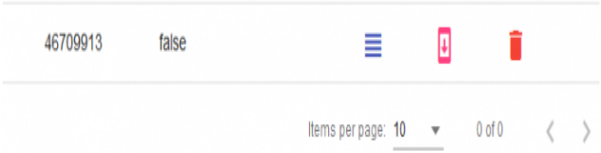
```
<mat-paginator [pageSize]="pageSize"
               [pageSizeOptions]="pageSizeOptions"
               (page)="pageEvent = $event">
</mat-paginator>
```

Somehow, it's showing 0 of 0 and can't filter out pageSize or pageSizeOptions.
Any idea?
Thanks.

Reply • Share ›

Kyle D ➚ Kyle D • 2 years ago

Forget to add picture . Lol



Reply • Share ›

Marinko Mod ➚ Kyle D • 2 years ago

It seems to me that you didn't provide a data source for your paginator. Can you check that, and if this doesn't solve issue, try comparing the source code with your solution.

Reply • Share ›

Kyle D ➚ Marinko • 2 years ago

I did provide datasource for pagination.

```
ngAfterViewInit(): void {
this.dataSource.sort = this.sort;
this.dataSource.paginator = this.paginator;
}
```

Code is the same. Triple check.
IS there any thing else need consider? Is the size(records) of table matter? I have like 35000 record in the table that I need to do pagination.

Reply • Share ›



Marinko Mod ➚ Kyle D • 2 years ago

Are you sure that yiu are fetching any data from the server at all? About that amount of data, I am not sure that there are restrictions but you can use our source code to check that.

^ | v • Reply • Share ›



Kyle D ➚ Marinko • 2 years ago

Yep. I'm able to get data from server at all. I'm not sure, but maybe the amount of data is causing this issue. I uses Postman to check how server return data and the size is 52 Mb.
Do you think that I need to limit how much data send to Angular from backend? Any guide line to read and follow? If I can do it, then I can verify if the amount of data is matter.
Any other suggest solutions ?
Thank a lot.

^ | v • Reply • Share ›



Marinko Mod ➚ Kyle D • 2 years ago

Well if you want to test that with fewer data, then instead of returning all, use some filter or the Take operator if you are using .net as a server side. Then you can see if reduced amount of data solves your problem.

^ | v • Reply • Share ›

© Copyright code-maze.com 2016 - 2021